1. Оператор select. Секции from, where, group by, having, order by, distinct, top.

Оператор `SELECT` — это мощный инструмент фильтрации и агрегации. Порядок написания секций строго регламентирован:

1. **SELECT**: Какие колонки берем?
   - `DISTINCT` — убирает дубликаты.
   - `TOP (N)` — берет первые N строк.
2. **FROM**: Из какой таблицы?
3. **WHERE**: Фильтрация конкретных строк (например, `Age > 18`).
4. **GROUP BY**: Группировка данных для расчетов (например, по отделам).
5. **HAVING**: Фильтрация уже *сгруппированных* данных (например, «оставить только отделы, где средняя зарплата > 50к»).
6. **ORDER BY**: Сортировка (`ASC` — по возрастанию, `DESC` — по убыванию).

Пример комплексного запроса:

SQL

```
SELECT TOP 5 Department, AVG(Salary) as AvgSalary
FROM Employees
WHERE Status = 'Active'
GROUP BY Department
HAVING AVG(Salary) > 1000
ORDER BY AvgSalary DESC;
```

---

2. Подзапросы и логические конструкции

Подзапрос — это запрос внутри другого запроса. Они помогают фильтровать данные на основе динамических условий.

- **IN**: Проверяет, входит ли значение в список (или результат подзапроса).
  - `WHERE ID IN (SELECT ManagerID FROM Departments)`
- **EXISTS**: Проверяет, возвращает ли подзапрос *хотя бы одну* строку. Это быстрее для больших таблиц, чем `IN`.
- **ALL**: Условие истинно, если оно выполняется для **всех** значений подзапроса.
- **ANY / SOME**: Условие истинно, если оно выполняется хотя бы для **одного** значения.
- **NOT**: Инвертирует результат (`NOT IN`, `NOT EXISTS`).

---

3. Группировка и Агрегатные функции

Агрегатные функции вычисляют одно значение на основе набора строк.

- **SUM()** — сумма.
- **AVG()** — среднее.
- **COUNT()** — количество строк (или не-NULL значений).
- **MIN() / MAX()** — минимум и максимум.

Группировка (`GROUP BY`) схлопывает строки с одинаковыми значениями в одну, чтобы применить к ним агрегат.

---

4. Расширенная группировка: CUBE и ROLLUP

Это специальные операторы для создания иерархических отчетов и промежуточных итогов.

- **ROLLUP**: Создает итоги «сверху вниз» по иерархии. Если группируем по `Страна, Город`, ROLLUP выдаст:
  1. Детализацию (Страна + Город).
  2. Итог по каждой Стране.
  3. Общий итог по всей таблице.
- **CUBE**: Генерирует **все возможные** комбинации итогов. Для `Страна, Город` он добавит еще и итог по всем Городам (независимо от страны).

5. Объединение таблиц (JOINs)

JOIN позволяют склеивать данные из разных таблиц по общему ключу.

Основные отличия:

Тип JOIN	Результат
INNER JOIN	Только те строки, которые имеют **совпадения в обеих** таблицах.
LEFT OUTER JOIN	**Все** строки из левой таблицы + совпадения из правой. Если в правой пусто — будет `NULL`.
RIGHT OUTER JOIN	**Все** строки из правой таблицы + совпадения из левой.
FULL OUTER JOIN	Все строки из обеих таблиц. Где нет пары — там `NULL`.
CROSS JOIN	«Декартово произведение». Каждая строка первой таблицы соединяется с каждой строкой второй (результат: $N \times M$ строк).

Пример (кто в каком отделе):

SQL

```
SELECT E.Name, D.DeptName
FROM Employees E
LEFT JOIN Departments D ON E.DeptID = D.ID;
-- Даже если у сотрудника нет отдела, он попадет в список (с NULL в DeptName).
```

Сводная таблица логики JOIN

Представьте две таблицы: **A** (Сотрудники) и **B** (Отделы).

- **INNER**: Сотрудники с отделами.

- **LEFT**: Все сотрудники (даже без отделов).
- **RIGHT**: Все отделы (даже пустые).
- **CROSS**: Каждому сотруднику приписываются все возможные отделы.

Хотите рассмотреть оконные функции (например, `ROW_NUMBER` или `RANK`), которые позволяют делать аналитику без «схлопывания» строк в `GROUP BY`?